# Estructura del lenguaje javascript

## Caso práctico



Ministerio de Educación (Uso educativo nc)

En **BK Programación** han decidido ponerse manos a la obra y **Antonio** comienza a estudiar los **fundamentos del lenguaje JavaScript**, (siempre bajo la tutoría de **Juan**, que estará ahí para ayudarle en todo momento).



Ministerio de Educación (Uso educativo nc)

En la unidad anterior **Antonio** analizó las posibilidades de los lenguajes de script, decidió qué lenguaje emplearía para la programación en el entorno cliente y vio cómo insertar en la página HTML dicho lenguaje de script.

Cómo han decidido emplear el lenguaje Javascript, lo primero que hará **Antonio** es ver los fundamentos de dicho lenguaje, y cuál es la estructura básica para comenzar a programar lo más pronto posible.

**Ada** está muy entusiasmada con este proyecto y está convencida de que **Antonio** será capaz de hacer ese trabajo sin ningún problema.

En esta unidad se hace una introducción a los fundamentos de JavaScript. Para ello se hará hincapié en la definición de variables, los tipos de datos soportados en JavaScript, conversiones, los operadores y terminaremos con las estructuras de control y bucles.

Al final de la unidad realizaremos nuestro primer ejemplo de JavaScript poniendo en práctica algunos de los contenidos aquí vistos.

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, FORMACIÓN PROFESIONAL Y DEPORTES

Ministerio de Educación, Formación Profesional y Deportes (Dominio público)

**Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes**

Aviso Legal

# 1.- Fundamentos de javascript

## Caso práctico



Ministerio de Educación (Uso educativo nc)

**Juan** habla con **Antonio** y le recomienda leer las bases y fundamentos de JavaScript, en los cuáles se explica cómo se ponen comentarios en el código, cómo se definen variables, estructuras de control, etc.

Éste va a ser uno de los primeros pasos que **Antonio** tendrá que dar para el estudio de cualquier lenguaje de programación, y en especial de JavaScript.

Como bien ya conocéis, una página web tiene tres capas:

- HTML (estructura): es el lenguaje que define la estructura para dar un sentido al contenido web.
- CSS (presentación):es el lenguaje utilizado para formatear cada uno de los elementos o grupos de elementos.
- JavaScript (comportamiento): Es un lenguaje de programación que te permite controlar el comportamiento de tu página web. ¡No sabes lo que podrás lograr utilizando JavaScript!

Antes de empezar con JavaScript, vamos a revisar los orígenes de JavaScript. Por ello, te muestro esta línea de tiempo en la que se incluyen los principales hitos en el desarrollo y estandarización de JavaScript.

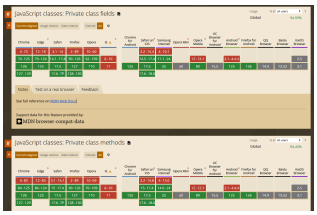
Mostrar

**Evolución de JavaScript**

- **Orígenes**
- **Versiones antiguas de EcmaScript**
- **Versiones nuevas de EcmaScript**

ECMAScript no para de evolucionar y los navegadores modernos rápidamente incorporan las nuevas funcionalidades definidas en el estándar. Por ejemplo, las propiedades y métodos privados ya están disponibles en todos los navegadores modernos.

Además, es posible convertir código JavaScript moderno a código JavaScript compatible con más navegadores con compiladores como Babel.



Óscar Gómez. *CanIUse. Comprobación de compatibilidades de JavaScript.* (Dominio público)

# Autoevaluación

**¿Quién desarrolló JavaScrip?**

- ◯ Microsoft.
- ◯ Oracle Corporation.
- ◯ Brendan Eich.

No es correcta, porque Microsoft creó JScript el cuál está basado en JavaScript.

Incorrecta, porque Oracle es la empresa propietaria de la licencia de JavaScript actualmente.

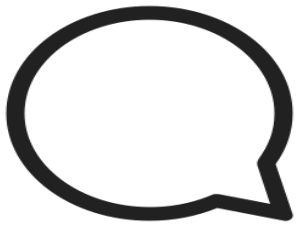
Muy bien. El nombre original fue Mocha, luego LiveScript, para pasar a ser finalmente JavaScript.

## Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

# 1.1.- Comentarios en el código

A la hora de programar en cualquier lenguaje de programación, es muy importante que comentes tu código. Aunque hoy día se aconseja que el propio código sea el que esté autodocumentado. Es decir, si una serie de sentencias realizan una determinada acción o función, es mejor y más claro crear una función para dicho propósito con un identificador lo suficientemente claro, que poner el comentario en el código. Pero todo eso lo veremos más adelante.



Lycopene579 (Dominio público)

Los comentarios son sentencias que el intérprete de JavaScript ignora. Sin embargo, estas sentencias permiten a los desarrolladores dejar notas sobre cómo funcionan las cosas en sus scripts.

Los comentarios ocupan espacio dentro de tu código de JavaScript, por lo que cuando alguien se descargue vuestro código necesitará más o menos tiempo, dependiendo del tamaño de vuestro fichero.

JavaScript permite dos estilos de comentarios: de una sola línea o de varias. Ejemplos de comentarios de una única línea o de varias líneas:

```
// Mi primer comentario

/*
    Mi segundo comentario,
    éste un poco más largo.
*/
```

## Recomendación

Si quieres comentar código JavaScript en Visual Studio Code, puedes hacerlo con el atajo de teclado **CTRL + Ç**.

# 1.2.- Consola y cajas de diálogo

## Consola

Para mostrar retroalimentación a través de la consola del navegador (accesible desde herramientas del desarrollador -> Consola) se utilizan las instrucciones:

**Console.log() - Muestra un mensaje por el log**

```
console.log("Hola mundo!!!");
console.log(1+1);
```

**Console.warn() - Muestra mensajes de aviso por la consola**

```
console.warn("Un aviso. Ten cuidado con x");
```

**Console.error() - Muestra mensajes de error por la consola**

```
console.error("Algo ha ido mal.");
```

### Para saber más

Existen muchos otros métodos para interactuar con la consola. Puede verse un resumen de los más importantes en: https://dev.to/devsmitra/javascript-console-methods-a-cheat-sheet-for-developers-44mc

## Cajas de diálogo (modales)

En JavaScript es también posible mostrar mensajes en cajas de diálogo, o habitualmente conocidos como modales, mediante las siguientes instrucciones:

**Alert() - Muestra un mensaje y espera a que se presione Aceptar**

```
alert("Hola Mundo!!");
```

### Promt() - Pide al usuario que introduzca un valor

```
let name= prompt ('¿Cómo te llamas?', "nombre por defecto");

alert(`Te llamas ${name}`);
```

### confirm() - Pide al usuario que confirme si quiere que se realice una acción

```
let isAdult = confirm("¿Eres mayor de edad?");

if(isAdult) console.log("Eres un adulto");
else console.log("No eres un adulto");
```

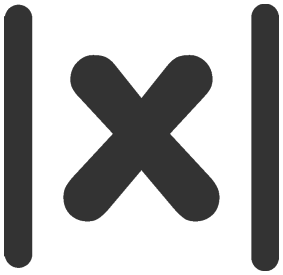
## Para saber más

Habitualmente se crean componentes en la propia página para actuar como modales, en lugar de los proporcionados directamente por el navegador, para poder darle el estilo que queramos. Aunque estos modales propios requieren conocimientos sobre el uso del DOM que veremos en próximas unidades.

Puedes ver un pequeño adelanto en:

- ¿Cómo hacer un modal CSS? (inglés).
- Cómo crear un modal emergente con JavaScript desde cero.

# 1.3.- Constantes y variables



Clker-Free-Vector-Images (Dominio Público)

Una **variable** es un contenedor de un valor de un tipo dado al que le damos un identificador. El valor de una variable puede cambiar en el tiempo. También existen las **constantes** que, al contrario de las variables, su valor no puede cambiar.

Para nombrar una variable o constante sólo podemos utilizar caracteres alfanuméricos `[a-zA-Z0-9]` y el carácter de subrayado `_`. No debe comenzar por un número. Tampoco podemos utilizar como identificador una palabra reservada ya que nos daría error. Es conveniente que los nombres, o los identificadores que les damos, sean descriptivos e informen del cometido de dicha variable para que nuestro código sea legible y entendible. También es conveniente seguir la convención lower camel case en la que se utilizan minúsculas para nombrarlas y si es la unión de varias palabras éstas se unen y se distingue entre palabras utilizando la primera letra en mayúscula.

Antes de utilizar una variable deberemos declararla y luego darle un valor, ya que si no su valor será **undefined**.

Para declarar constantes y variables debemos utilizar las palabras reservadas **const**, **var** y **let** (la primera para las constantes y las otras para las variables en las que ahora nos detendremos).

```
const pi = 3.14;
var edad = 18;
let nombre = "Bob Esponja";
```

La palabra **var** declara una variable y se sigue utilizando por motivos históricos ya que genera algunos problemas que viene a solucionar **let**.

Debido al **hoisting**, era posible declarar una variable con **var** después de ser utilizada y eso funcionaba. Por ejemplo:

```
edad = 18;
...
...
var edad;
```

Ese código era válido, pero la verdad es que se hace confuso y más complejo de entender. Pues el **hoisting** ya no funciona con **let**, por lo que para utilizar una variable que vamos a declarar con **let**, primero debemos declararla o de lo contrario nos dará un error.

```
let edad;
...
edad = 18;
```

Además, con el uso de **var**, una variable se podía declarar varias veces y era correcto, pero como ves en el ejemplo vuelve a ser confuso.

```
var edad = 18;
....
var edad = 20;
```

Eso con `let` ya no funciona y habría que hacerlo de la siguiente forma:

```
let edad = 18;
....
edad = 20;
```

JavaScript es un lenguaje "débilmente tipado", lo cual significa que, a diferencia de otros lenguajes, no es necesario especificar el tipo de dato que vamos a almacenar en una variable y además éste puede cambiar. Esto deberíamos evitarlo **para** prevenir posibles confusiones.

```
let edad;
edad = '18';    // En este caso contiene una cadena
edad = 18;      // En este caso contiene un número
```

| | Ámbito global | Ámbito función | Ámbito bloque | Reasignable | Redeclarable | Hoisted (declaración después de uso) |
|---|---|---|---|---|---|---|
| **var** | Sí | Sí | No | Sí | Sí | Sí |
| **let** | No | Sí | Sí | Sí | No | No |
| **const** | No | Sí | Sí | No | No | No |

## Autoevaluación

**Para definir una variable hoy día es más conveniente utilizar `var` que `let`. ¿Verdadero o falso?**

○ Verdadero ○ Falso

**Falso**

Es mejor que nos acostumbremos a utilizar siempre `let` por las ventajas que hemos mencionado.

## Debes conocer

En JavaScript, el nombrado de las variables y métodos se hace siguiendo la nomenclatura CamelCase. Existen dos tipos:

- **Lower Camel Case:** Se utiliza principalmente para nombrar variables y funciones. La primera palabra se escribe completamente en minúsculas, y las palabras siguientes comienzan con una letra mayúscula. Ejemplos:

```
// Lower Camel Case (variables y funciones)
let perroNombre="Kira";
let perroEdad=5;
function holaMundo(){   console.log("Hola Mundo!");
```

- **Upper Camel Case**: Se utiliza comúnmente para nombrar clases y constructores en JavaScript. En este caso, la primera letra de cada palabra se escribe en mayúscula. Ejemplos:

```
// Upper Camel Case (clases)
class Perro {};
```

Además, el uso del símbolo $ como prefijo para nombrar variables que hacen referencia a nodos del DOM es una convención común en JavaScript, especialmente en el desarrollo web, aunque no es un requisito del lenguaje. Esta convención ayuda a los desarrolladores a identificar rápidamente que una variable específica está relacionada con un elemento del DOM y no con otro tipo de dato. Por ejemplo, en un código JavaScript:

```
const $nombre = document.getElementById('nombre');
const $botonEnviar = document.querySelector('.btn-enviar');
```

Puedes profundizar en:

- Técnicas de nombrado con CamelCase.
- Una guía de convenciones de nombrado en JavaScript (inglés).

# 1.4.- Tipos de datos

Las variables en JavaScript podrán contener cualquier tipo de dato. A continuación, se muestran los tipos de datos soportados en JavaScript:



Everaldo Coelho (GNU/GPL)

- **Números**: Pueden almacenar tanto números enteros como números decimales. Para expresar el valor de un número decimal se utiliza el separador **punto(.)**.
  - Si se quiere comprobar si un dato es entero se utiliza: **Number.isInteger()**
  - Si se quiere comprobar si un dato es un número se utiliza **isNaN()**
- **BigInt**: valor numérico grande.
- **Cadenas**: Las cadenas de texto son una secuencia de caracteres. Para expresar su valor, éstas se pueden encerrar entra comillas simples ' o comillas dobles ".
- **Valores lógicos**: Pueden almacenar los valores **true** o **false** y se suelen utilizar en las sentencias condicionales o ser el resultado de una operación de comparación como luego veremos.
- **Arrays**: son objetos que contienen diferentes valores, que pueden ser de diferentes tipos, indexados por su índice, comenzando en 0. Para declararlos utilizamos los corchetes y para acceder a sus valores utilizamos su índice encerrado entre corchetes:

```
let personajes = ['Bob Esponja', 'Calamardo', 'Patricio'];
console.log(personajes[0]);        // Muestra por consola: Bob Esponja
```

  Si se usa **typeof** con un array devuelve que es del tipo objeto. Si se quiere comprobar si un dato contiene un array se debe utilizar **Array.isArray([])**.
- **Objetos**: Un objeto no es más que una colección de atributos, cada uno de ellos asociado a un valor. Para definir un objeto utilizamos las llaves y expresamos cada uno de sus atributos indicando el nombre y su valor separados por dos puntos. Los objetos en JavaScript no tienen por qué ser instancias de clases.

```
let personaje = { nombre: 'Bob Esponja', edad: 5 };
console.log(personaje.nombre);      // Muestra por consola: Bob Esponja
console.log(personaje[nombre]);     // Muestra por consola: Bob Esponja
```

  El literal {} representa un objeto vacío (sin atributos)

```
const y = {};
```

- **Funciones**: Las funciones permiten ejecutar bloques de instrucciones y también se pueden almacenar en variables.

## Autoevaluación

**Los arrays en javascript comienzan en índice 1. ¿Verdadero o falso?**

◯ Verdadero ◯ Falso

**Falso**

Los arrays en JavaScript comienzan en el índice 0.

## Debes conocer

Con el comando `typeof` puedes comprobar el tipo de un dato de una variable:

```
//Source: <a href="https://www.w3schools.com/js/js_typeof.asp" target="_blank" rel="noopene
typeof "John"          // Returns string
typeof ("John"+"Doe") // Returns string
typeof 3.14            // Returns number
typeof 33              // Returns number
typeof (33 + 66)       // Returns number
typeof true            // Returns boolean
typeof false           // Returns boolean
typeof 1234n           // Returns bigint
typeof Symbol()        // Returns symbol
typeof x               // Returns undefined

typeof null            // Returns object

typeof []               // Returns object
Array.isArray([])       // Returns true
```

## Para saber más

JavaScript presenta problemas de precisión cuando se trabaja con números en coma flotante (números reales). El ejemplo más habitual es:

```
console.log(0.1+0.2); // return 0.30000000000000004
```

Puedes profundizar más sobre el tema en los siguientes enlaces:

- Hilo de twitter de Manz.
- ¿Por qué 0.1 +0.2 es diferente a 0.3 en JavaScript?

# 1.4.1.- Conversiones de tipos de datos



Everaldo Coelho (GNU/GPL)

Aunque los tipos de datos en JavaScript son muy sencillos, a veces te podrás encontrar con casos en los que las operaciones no se realizan correctamente, y eso es debido a la conversión de tipos de datos. JavaScript intenta realizar la mejor conversión cuando realiza esas operaciones, pero a veces no es el tipo de conversión que a ti te interesaría.

Por ejemplo, cuando intentamos sumar dos números:

```
console.log(4 + 7);    // Muestra por consola: 11
```

Si uno de esos números está en formato de cadena de texto, JavaScript lo que hará es intentar convertir el otro número a una cadena y los concatenará, por ejemplo:

```
console.log(4 + '7');     // Muestra por consola: 47 (como cadena)
```

Otro ejemplo podría ser:

```
console.log(1 + 2 + '3');     // Muestra por consola: 33 (como cadena)
```

Esto puede resultar ilógico, pero sí que tiene su lógica. La expresión se evalúa de izquierda a derecha. La primera operación funciona correctamente devolviendo el valor de 3 pero al intentar sumarle la cadena de texto 3, JavaScript lo que hace es convertir ese número a una cadena de texto y se lo concatenará al otro 3.

Para convertir cadenas a números dispones de las funciones: **parseInt()** y **parseFloat()**.

Por ejemplo:

```
console.log(parseInt('34'));              // Muestra por consola: 34
console.log(parseInt('89.76'));          // Muestra por consola: 89
console.log(parseFloat('34'));           // Muestra por consola: 34
console.log(parseFloat('3.14'));        // Muestra por consola: 3.14
console.log(1 + 2 + parseInt('3'));     // Muestra por consola: 6
```

Si lo que deseas es realizar la conversión de números a cadenas, es mucho más sencillo, ya que simplemente tendrás que concatenar una cadena vacía al principio, y de esta forma el número será convertido a su cadena equivalente:

```
console.log('' + 2020);        // Muestra por consola: 2020 (como cadena);
```

Cuando la conversión se realiza automáticamente es **conversión implícita**. Mientras la que se realiza mediante una instrucción es **conversión explícita**.

# Para saber más

A continuación tienes una serie de conversiones implícitas aplicadas en diferentes contextos. Te animo a revisar con calma el porqué del resultado de cada una de ellas:

```
"3" * 4// → 12
3 * 4// → 12
"3" * "4"// → 12
"3" * "pepe"// → NaN
"12" + "20"// → 1220
"12" + 20// → 1220
12 + "20"// → 1220
Math.log10("1000")// → 3
"10" < "2"// → true
"10" < 2// → false
```

```
if (23) { .... }// → se cumple
if (-1) { .... }// → se cumple
if (0) { .... }// → no se cumple
if ("Pepe") { .... }// → se cumple
if ("") { .... }// → no se cumple
if ([1, 3]) { .... }// → se cumple
if ([]) { .... }// → se cumple
if (null) { .... }// → no se cumple
if (undefined) { .... }// → no se cumple
```

```
"25" == 25// → true
"25" === 25// → false
false == 0// → true
"" == 0// → true
2.0 === 2 // → true (recuerda: no se distingue entre tipo de enteros y de coma flotante)
```

# 1.5.- Operadores



Everaldo Coelho (GNU/GPL)

JavaScript es un lenguaje rico en operadores: símbolos y palabras que realizan operaciones sobre uno o varios valores, para obtener un nuevo valor.

Cualquier valor sobre el cuál se realiza una acción (indicada por el operador), se denomina un operando. Una **expresión** puede contener un operando y un operador (denominado operador unario), como por ejemplo en `b++`, o bien dos operandos, separados por un operador (denominado operador binario), como por ejemplo en `a + b`. Incluso existe algún operador ternario, como luego veremos.

## Categorías de operadores en JavaScript

| Tipo | Qué realizan |
|---|---|
| Comparación. | Comparan los valores de 2 operandos, devolviendo un resultado de `true` o `false` (se usan extensivamente en sentencias condicionales.<br><br>`==  !=  === !==  >  >=  <  <=`<br><br>En JavaScript, == compara valores con conversión de tipos, mientras que === compara tanto el tipo como el valor sin conversión. Por lo tanto, === es más seguro para evitar resultados inesperados.<br><br>`1=="1"  //devuelve true`<br>`1==="1" //devuelve false` |
| Aritméticos. | Unen dos operandos para producir un único valor que es el resultado de una operación aritmética u otra operación sobre ambos operandos.<br><br>`+  -  *  /  %  ++  - -  +valor  -valor` |
| Asignación. | Asigna el valor a la derecha de la expresión a la variable que está a la izquierda.<br><br>`=  +=  -=  *=  /=  %=  <<=  >=  >>=  >>>=  &=  \|=  ^ =  []` |
| Lógicos. | Realizan operaciones lógicas sobre uno o dos operandos lógicos.<br><br>`&&  \|\|  !` |
| Bit a Bit. | Realizan operaciones aritméticas o de desplazamiento de columna en las representaciones binarias de dos operandos. |

| Tipo | Qué realizan |
| --- | --- |
| | `&` `\|` `^` `~` `<<` `>>` `>>>` |
| Objeto. | Ayudan a los scripts a evaluar la herencia y capacidades de un objeto particular antes de que tengamos que invocar al objeto y sus propiedades o métodos.<br>`.` `[]` `()` `delete` `in` `instanceOf` `new` `this` |
| Otros. | Operadores que tienen un comportamiento especial.<br>`,` `?:` `typeof` `void` |

## Debes conocer

En el siguiente enlace podrás encontrar más información sobre los operadores de JavaScript, por lo que no nos detendremos más en ellos, aunque sí veremos algunos ejemplos que merecen la pena destacar

Operadores en JavaScript

## Autoevaluación

¿Cuál de las siguientes comparaciones devuelve `false`?

- 1=="1"
- 1==1
- 1===1
- 1==="1"

Incorrecto

Incorrecto

Incorrecto

Opción correcta

## Solución

1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

# 1.5.1.- Algunos operadores de interés

### typeof



Everaldo Coelho (GNU/GPL)

Este operador unario se usa para identificar el tipo de una variable o expresión.

```
let indefinido;
let cadena = '5'
let numero = 5;
console.log(typeof indefinido);       // Muestra por consola: undefined
console.log(typeof cadena);           // Muestra por consola: string
console.log(typeof numero);           // Muestra por consola: number
```

### Plantillas de cadenas (template string)

Con ES6 podemos definir cadenas de una forma más sencilla que como estábamos haciendo hasta ahora. Fíjate en este ejemplo, en el que para utilizar la plantilla de cadenas utilizamos la comilla invertida ` y ponemos las variables encerradas entre **${}**.

```
let nombre = 'Bob Esponja';
console.log(`Hola ${nombre}`);      // Muestra por consola: Hola Bob Esponja
```

En ES6 también podemos definir cadenas en varias líneas, sin necesidad de utilizar el operador de concatenación **+**.

### Desestructuración

En ES6 también tenemos nuevas formas de asignar valores a variables a partir de un array.

```
let [personaje1, personaje2] = ['Bob Esponja', 'Calamardo'];
console.log(personaje1);       // Muestra por consola: Bob Esponja
console.log(personaje2);       // Muestra por consola: Calamardo
```

Esa misma desestructuración también la podemos hacer con objetos.

```
let personaje = { nombre: 'Bob esponja', edad: 18 };
let { nombre, edad } = personaje;
console.log(nombre);       // Muestra por consola: Bob Esponja
console.log(edad);         // Muestra por consola: 18
```

### ? :

Este operador condicional es la forma reducida de la expresión **if - else**.

La sintaxis formal para este operador condicional es:

**(condicion) ? expresionV : expresionF;**

Si la condición evalúa a **true** devuelve **expresionV** y en caso contrario devuelve **expresionF**.

```
let sexo = 'V';
```

```
console.log(`Su sexo es ${sexo ? 'varón' : 'hembra'}`);
```

### ?.

El operador de encadenamiento opcional ?. es la forma más segura de acceder a la propiedad de un objeto. Si no puede acceder a la propiedad de un objecto, devuelve null en lugar de disparar un error.

```
let persona={};
if(persona?.nombre)
{
    console.log(`El nombre de la persona es ${persona.nombre}`)
}
else{
    console.warn("El objeto persona no tiene la propiedad nombre");
}

//Devuelve: El objeto persona no tiene la propiedad nombre

let persona2={nombre: "Oscar"};
if(persona2?.nombre)
{
    console.log(`El nombre de la persona2 es ${persona2.nombre}`)
}
else{
    console.warn("El objeto persona2 no tiene la propiedad nombre");
}

//Devuelve: El objeto persona no tiene la propiedad nombre
```

### ??

El operador de fusión de null (nullish Coalescing) permite devolver un valor si su precedente es null. Se suele utilizar en combinación con el operador de encadenamiento opcional.

```
//Ejemplo sencillo
let edad;
console.log(edad??18);
/Muestra 18

//Ejemplo en combinación con el operador de encadenamiento nulo.
let persona={};
console.log(persona?.nombre ?? "oscar");
//Muestra oscar
```

# 1.6.- Condiciones y bucles



Everaldo Coelho (GNU/GPL)

En esta sección te mostraremos cómo los programas pueden tomar decisiones, y cómo puedes lograr que un script repita un bloque de instrucciones las veces que quieras.

Cuando te levantas cada día tomas decisiones de alguna clase; muchas veces ni te das cuenta de ello, pero lo estás haciendo. Por ejemplo, imagínate que vas a hacer la compra a un supermercado; desde el momento que entras en el supermercado ya estás tomando decisiones: ¿compro primero la leche o compro la verdura?, ¿ese precio es barato o es caro?, ¿el color de ese tinte es azul claro u oscuro?, ¿tengo suficiente dinero para pagar o no?, ¿me llegan estos kilogramos de patatas o no?, ¿pago en efectivo o con tarjeta?, etc.

Otras veces repites un ejercicio físico un número determinado de veces para estar en forma. O realizas cambios a un documento hasta que crees que está perfecto. O preguntas a tus amigos mientras uno no te responda que él es el que tiene ese libro que tanto querías leer.

Es decir, tomamos innumerables decisiones a lo largo del día y la mayor parte de las veces no nos damos ni cuenta de ello. Pero también realizamos acciones repetitivas sin casi darnos ni cuenta.

En las siguientes secciones, verás cómo puedes ejecutar unas u otras instrucciones, dependiendo de ciertas condiciones, y cómo puedes repetir una o varias instrucciones, las veces que te hagan falta.

# 1.6.1.- Estructuras de control

En los lenguajes de programación, las instrucciones que te permiten controlar las decisiones y bloques de ejecución se denominan "Estructuras de Control". Una estructura de control dirige el flujo de ejecución a través de una secuencia de instrucciones, basadas en decisiones simples y en otros factores.



Peggy_Marco (Dominio Público)

Una parte muy importante de una estructura de control es la "condición". Cada condición es una expresión que se evalúa a **true** o **false**.

### Sentencia if

La decisión más simple que podemos tomar en un programa es la de seguir una rama determinada si una determinada condición evalúa a **true**.

```
if (condicion) {
      sentencias;
}
```

Si la condición evalúa a **true** se ejecutarán las sentencias dentro del bloque encerrado entre llaves.

```
if (edad >= 18) {
      alert('Ya eres mayor de edad');
}
```

### Sentencia if - else

En este tipo de construcción, decidimos si ejecutar una secuencia u otro en función de que la condición evalúe a **true** o **false**.

```
if (condicion) {
      sentenciasV;
} else {
      sentenciasF;
}
```

Si la condición evalúa a **true** ejecutamos **sentenciasV** y en caso contrario ejecutamos **sentenciasF**.

```
if (edad >= 18) {
      alert('Ya eres mayor de edad');
} else {
      alert('Aún eres menor de edad');
}
```

La estructura de control se puede extender a más de dos posibilidades con el uso de **else if**.

```
if (hora < 10) {
  saludo = "¡Buenos días!";
} else if (time < 20) {
  saludo  = "¡Buenas tardes!";
} else {
```

```
  saludo = "¡Buenas noches!";
}
```

Recuerda que también existía el operador ternario `?:` .

```
let nota=7;
console.log(nota < 5 ? "suspendido" : "aprobado");
```

### Sentencia `switch`

Por último, nos queda repasar la estructura de control switch. La declaración switch comprueba si el valor proporcionado corresponde a alguno de los casos (cases) que define y lo ejecuta, en caso contrario ejecuta el caso por defecto.

```
switch (mascota) {
  case "Perro":
    console.log("Tengo un perro de mascota.");
    break;
  case "Gato":
    console.log("Tengo un gato de mascota.");
    break;
  case "Canario":
  case "Loro":
    console.log("Tengo un pajaro de mascota.");
    break;
  default:
    console.log("No tengo mascota.");
}
```

## Autoevaluación

**Es obligatorio que en una sentencia `if` siempre utilicemos la claúsula `else`. ¿Verdadero o falso?**

◯ Verdadero ◯ Falso

**Falso**

La claúsula `else` puede estar presente o no.

# 1.6.2.- Bucles

Los bucles son estructuras repetitivas, que se ejecutarán un número de veces fijado expresamente, o que dependerá de si se cumple una determinada condición.



OpenClipart-Vectors (Dominio Público)

**Bucle for.**

Este tipo de bucle te deja repetir un bloque de instrucciones un número limitado de veces.

```
for (expresión inicial; condicion; incremente) {
      sentencias;
}
```

Primero se ejecuta la expresión inicial o de inicialización. En cada repetición del bucle se evalúa la condición y si ésta evalúa a **true**, se ejecutan las instrucciones, se ejecuta la sentencia de incremento y se repite el proceso de evaluación, ejecución e incremento hasta que la condición evalúe a **false**.

```
let numero = 5;
for (let i = 0; i <= 10; i++) {
      console.log(numero * i);
}
```

**Bucle while.**

Este tipo de bucles se utilizan cuando queremos repetir la ejecución de unas sentencias un número indefinido de veces, siempre que se cumpla una condición. Es más sencillo de comprender que el bucle **for**, ya que no incorpora en la misma línea la inicialización de las variables, su condición para seguir ejecutándose y su actualización. Sólo se indica, como veremos a continuación, la condición que se tiene que cumplir para que se realice una iteración o repetición.

```
while (condicion) {
      sentencias;
}
```

Se ejecutan las sentencias mientras la condición evalúe a **true**. Por tanto, dentro de dichas sentencias deberá darse la situación para que en algún momento la condición evalúe a **false** o de lo contrario estaríamos ante un **bucle infinito**.

```
let numero = 5;
let i = 0;
while (i <= 10) {
      console.log(numero * i);
      i++;
}
```

### Bucle do - while.

Este tipo de bucle es la última de las estructuras para implementar repeticiones de las que dispone JavaScript, y es una variación del bucle **while** visto anteriormente. Se utiliza generalmente, cuando no sabemos el número de veces que se habrá de ejecutar el bucle. Es prácticamente igual que el bucle **while**, con la diferencia, de que sabemos seguro que el bucle por lo menos se ejecutará una vez.

```
do {
      sentencias;
} while (condicion);
```

Al igual que en el caso anterior, siempre debe darse que las sentencias en el algún momento hagan que la condición evalúe a **false**, para evitar el bucle infinito.

```
let numero = 5;
let i = 0;
do {
      console.log(numero * i);
      i++;
} while (i <= 10);
```

También existen otras funciones para recorrer colecciones u objetos iterables que veremos más adelante (**foreach**, **map**, **filter**, **reduce**, …).

### Bucle for...of

El bucle **for...of** proporciona una forma sencilla de recorrer los elementos de un objecto interatable, como lo son el string, el array o el NodeList.

```
et iterable = [10, 20, 30];

for (let value of iterable) {
  console.log(value);
}
```

### Bucle for-in

El bucle **for...in** proporciona una forma sencilla de recorrer las propiedades de un objeto.

```
var obj = { a: 1, b: 2, c: 3 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
}

// Produce:
// "obj.a = 1"
// "obj.b = 2"
// "obj.c = 3"
```

Alternativamente, se puede conseguir una lista de todas las claves de un objeto con Object.keys(obj).

```
const persona= {
  nombre: 'Ana',
  edad: 22,
};

console.log(Object.keys(persona));
// Expected output: Array ["nombre", "edad"]
```

Otras funciones interesantes son Object.values(obj) para obtener un array con todos los valores y Object.entries(obj) para obtener un array de parejas clave-valor.

## Autoevaluación

**Un bucle for es imposible que se convierta en un bucle infinito ya que lleva implícita la sentencia de incremento. ¿Verdadero o falso?**

○ Verdadero ○ Falso

**Falso**

Eso depende de la condición y la sentencia de incremento y si no lo hacemos bien sí es posible que se convierta en un bucle infinito.

## Debes conocer

Se puede interrumpir el flujo normal de un bucle con las instrucciones continue, para saltar a la siguiente interacción del bucle, y break para terminar el bucle.

```
//Ejemplo de continue

for (let i = 0; i < 5; i++) {
  if (i === 3) { continue; }
  console.log(i);
}
//Muestra: 0, 1, 2 y 4


//Ejemplo de break

for (let i = 0; i < 5; i++) {
  if (i === 3) { continue; }
  console.log(i);
}
//Muestra: 0, 1 y 2
```

# 1.7.- Captura de errores

Otro aspecto clave de cualquier programa es la gestión de errores con try...catch. En el bloque del try se definen las instrucciones a ejecutar y en el del catch las instrucciones a ejecutar si hay alguna excepción. También se puede añadir un tercer bloque de finally para ejecutar siempre al final.

```
try{
    funcionNoExiste();
}
catch(e){
    console.error(e)
}
```

También es posible lanzar nuestros propios errores con throw

```
let num1=5;
let num2=0;

try{
    if(num2==0) throw "No se puede dividir por 0";
    console.log(`${num1} / ${num2} = ${num1/num2}`)
}
catch(e){
    console.error(e);
}
```

## Debes conocer

En programación, una **"guard clause"** es un patrón en el que se utilizan declaraciones condicionales `if` al inicio de una función o método para manejar rápidamente condiciones excepcionales o casos especiales, evitando así anidar el código en estructuras complejas y mejorando la legibilidad. La idea es retornar o salir de la función lo antes posible cuando se cumplan ciertas condiciones, dejando el flujo principal de la función libre de bifurcaciones innecesarias.

Por ejemplo, en lugar de anidar múltiples bloques if, podemos usar una guard clause para manejar los casos excepcionales de inmediato. La salida puede realizarse tanto con un `return` como con un `throw new Error("");`.

```
function procesarPedido(pedido) {
    if (!pedido) {
        throw new Error("El pedido no puede ser nulo.");
    }

    // Lógica principal para procesar el pedido
    console.log(`Procesando pedido: ${pedido}`);
}
```

## Para saber más

Puedes profundizar sobre la gestión de errores en: https://es.javascript.info/try-catch

# 1.8.- Funciones

Las funciones permiten agrupar un conjunto de instrucciones bajo un nombre específico, facilitando la reutilización de código y mejorando la organización y legibilidad del programa. Las funciones tradicionales se definen utilizando la palabra clave `function` seguida del nombre de la función y un conjunto de paréntesis que pueden incluir parámetros (si no se pasan su valor será `undefined`). Dentro de las llaves `{}` se escriben las instrucciones que componen la función.

```
function saludar(nombre) {
    console.log(`Hola, ${nombre}!`);
}

saludar("Carlos"); // Salida: Hola, Carlos!
```

Las funciones en JavaScript tienen mucha más profundidad. Desde la posibilidad de pasar parámetros predeterminados o almacenar una función en una variable o incluso pasarla por parámetro (a veces llamado callback en el contexto del desarrollo web, o funciones de orden superior), hasta maneras alternativas de definir funciones como son las funciones anónimas y las funciones flecha. En futuras unidades profundizaremos sobre estos conceptos.

## Autoevaluación

Considerando la función `saludar(nombre)`, ¿qué sucede si llamas a la función saludar sin pasarle un parámetro?

- ○ La función arroja un error porque falta un parámetro obligatorio.
- ○ La función imprime "Hola, undefined!" en la consola.
- ○ La función no realiza ninguna acción y no produce salida.
- ○ La función imprime "Hola, !" sin el nombre en la consola.

Incorrecto

Opción correcta

Incorrecto

Incorrecto

# Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

# 1.9.- Primeros ejercicios con JavaScript



Everaldo Coelho (GNU/GPL)

En este apartado vamos a ver varios ejemplos sencillos de uso de JavaScript para hacer páginas webs interactivas. En los últimos ejemplos hay alguna sintaxis que aún no hemos presentado, pero es fácil y entendible. La idea es que vayas familiarizándote con el entorno que utilices, con la forma de trabajar y sobre todo con la forma de pensar.

Si hay alguna cosa que aún no entiendas no te desesperes, pregunta en el foro, que encantado te responderé a tus dudas. Pero lo que sí quiero es que te centres en las cosas que ya conoces y cómo y por qué las he utilizado. Os animo a todos y todas a que implementéis este ejemplo, utilizando el editor o entorno de vuestra preferencia, lo visualices en el navegador o navegadores que tenéis instalados y que vayáis familiarizándoos con todo.

# 1.9.1.- Acceso y modificación de un elemento del DOM

Para crear una web interactiva, lo primero es ser capaz de acceder y modificar sus elementos. Para obtener un nodo por su ID, se utiliza `document.getElementById("id")` o `document.querySelector("#id")`. Cabe notar que con `querySelector` se deben usar selectores CSS para identificar el nodo que se busca.

Por ejemplo, dado el siguiente código HTML con un nodo con el ID "modifyMe";

```
<html>
    <body>
        <p id="modifyMe"></p>
        <script src="./scripts/main.js" defer></script>
    </body>
</html>
```

La mejor forma más sencilla de modificar este elemento es:

```
document.getElementById("modifyMe").innerHTML="Hola Mundo!";
```

Por último, las instrucciones de inicialización de la página suelen agruparse en una función `init()` que es llamada cuando la página está cargada con el evento de OnLoad. Para hacer que la función init se llame cuando la página esté cargada se utiliza `window.addEventListener("load",init)` ó `window.onload=init`;

```
window.addEventListener("load",init);
//window.onload=init;

function init(){
    document.getElementById("modifyMe").innerHTML="Hola Mundo!";
}
```

JavaScript es sensible a las mayúsculas por lo que tienes que tener cuidado de escribir los métodos correctamente.

```
document.getElementById("modifyMe").innerHTML="Hola Mundo!"; //Correcto
document.getElementById("modifyMe").innerhtml="Hola Mundo!"; //Genera error
```

## Autoevaluación

`document.getElementById("modifyMe").innerHTML = "Nuevo texto"` se utiliza para cambiar el contenido de un elemento HTML con el ID "`modifyMe`".

○ Verdadero ○ Falso

**Verdadero**

# 1.9.2.- Botones

Una de las primeras formas de proporcionar interactividad a una web es mediante los botones, o básicamente cualquier elemento que reaccione al evento de *click* sobre él. Una primera forma de hacer un elemento llamar a código JavaScript cuando se le hace click es añadiendole el atributo onClick.

```
<!DOCTYPE html>
<html>
<body>

<p id="demo">Mi primera web interactiva.</p>

<button id="boton" type="button" onclick='document.getElementById("demo").innerHTML = "Hola mundo!"'>¡Hazl

</body>
</html>
```

Sin embargo, poner el código JavaScript directamente en el código HTML es considerado una mala práctica dado que se está mezclando la capa de estructura con la de comportamiento. Por ello se recomienda enlazar los eventos directamente desde el código JavaScript. En futuras unidades seguiremos profundizando sobre eventos, de momento nos basta con aprender que si obtenemos un elemento con document.getElementByID() podemos añadir una reacción cuando el usuario haga click con **addEventLitener("click",nombreFunción)**;

```
document.getElementById("boton").addEventListener("click",funcionClick);

function funcionClick(){
    document.getElementById("demo").innerHTML = "Hola mundo!";
}
```

Cuando se quiere llamar a una función JavaScript desde el HTML, dentro del atributo **onClick**, se debe usar **miFuncion()** **con paréntesis**. Sin embargo, cuando se asocian funciones a eventos desde JavaScript con **addEventListener** u otro método, se debe poner **sin paréntesis**. De lo contrario, en lugar de llamar a la función cuando se haga click, llamaría al resultado de la función.

## Para saber más

Puedes ver otros ejemplos interesantes de usos básicos de JavaScript en los siguientes enlaces:

- Uso de JavaScript para cambiar estilo (w3schools).
- Ocultar elementos (w3schools).
- Cambiar imagen (w3school).

# Para saber más

El código del ejemplo se puede hacer de forma aún más organizada, permitiendo a la larga una mayor control y escalabilidad de nuestra aplicación.

```
//Declaración de variables
let textoNodo;
let botonNodo;

//Inicialización
window.addEventListener("load",init);

function init(){
    //Obtengo las referencias a los nodos. Es más eficiente que realizar siempre la búsque
    textoNodo=document.getElementById("demo");
    botonNodo=document.getElementById("boton");

    //Asocio eventos
    botonNodo.addEventListener("click",funcionClick);
}

//Funciones de respuesta
function funcionClick(){
    textoNodo.innerHTML = "Hola mundo!";
}
```

No te preocupes si este último código te parece demasiado complejo. A lo largo del curso irás acostumbrándote al lenguaje y profundizando sobre los diferentes elementos que se han presentado "rápidamente aquí" como el concepto de nodo o evento.

# 1.9.3. Obtener valores de inputs

Si se accede al nodo de un elemento HTML de tipo input, se puede acceder a su valor inicial con la propiedad **.value**. Por ejemplo, si se tiene el siguiente HTML:

```
<label for="nombre">Nombre:</label>
<input id="nombre" type="text" />
<button id="boton" type="button">¡Saludame!</button>
<p id="demo"></p>
```

El código JavaScript para hacer que al hacer click el párrafo con id demo se rellene con "Hola <nombre>" es:

```
document.getElementById("boton").addEventListener("click",saludar);

function saludar(){
    let nombre=document.getElementById("nombre").value;
    document.getElementById("demo").innerHTML = `Hola ${nombre}!`;
}
```

## Para saber más

Existen otros eventos a los que se puede reaccionar. Por ejemplo, si se quiere que se llame a una función cada vez que se modifique el valor del campo de texto basta con utilizar el evento input del nodo de input.

## Para saber más

El código anterior se puede realizar de forma más estructura de la siguiente forma:

```
//Declaro las variables
let textoNodo;
let inputNodo;
let botonNodo;

//Inicialización
window.addEventListener("load",init);

function init(){
    //Obtengo las referencias a los nodos
    textoNodo=document.getElementById("demo");
    inputNodo=document.getElementById("nombre");
```

```
    botonNodo=document.getElementById("boton");

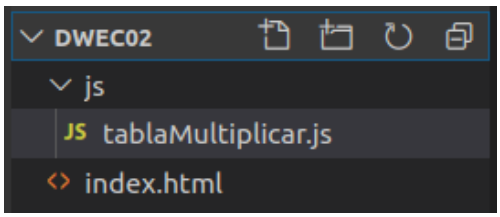
    //Asocio eventos
    botonNodo.addEventListener("click",saludar);
}

//Funciones de respuesta
function saludar(){
    textoNodo.innerHTML = `Hola ${inputNodo.value}!`;
}
```

# 1.9.4.- Tabla de multiplicar

Un formulario en el que se pide al usuario que introduzca un número entre 0 y 10, ambos inclusive, y una vez que envíe el formulario le muestra la tabla de multiplicar para dicho número introducido.

Veamos primero cómo he estructurado el proyecto.




Visual Code Studio (Todos los derechos reservados)

Una vez vista la estructura, pasemos a ver el formulario y la inclusión de nuestro fichero **.js** en el archivo **index.html**.

```
<!DOCTYPE html>
<html lang="es-ES">
    <head>
        <meta charset="UTF-8">
        <title>Tabla de multiplicar</title>
        <script src="js/tablaMultiplicar.js"></script>
    </head>
    <body>
        <h1>
            Tabla de multiplicar
        </h1>
        <form onsubmit="mostrarTabla()">
            <label for="numero">Introduce el número:</label>
            <input id="numero" type="number" required/>
            <input type="submit" value="Mostrar tabla" />
        </form>
        <div id="tabla">
        </div>
    </body>
</html>
```

Como podéis apreciar, simplemente incluimos el archivo **tablaMultiplicar.js** para que su contenido esté accesible. Creamos la estructura del documento, en la que básicamente creamos dos elementos:

- Un formulario para que el usuario introduzca el número del que quiere visualizar su tabla de multiplicar. Este formulario, cuando es enviado (aunque no es enviado a ningún lado, simplemente se valida en lado del cliente), mediante el evento **onSubmit** del mismo, simplemente llama a la función **mostrarTabla()** que está definida en nuestro archivo **tablaMultiplicar.js**.
- Un bloque vacío cuyo **id** es **tabla** y en el que luego mostraremos los resultados.

Por otro lado, la función **mostrarTabla()** del archivo **tablaMultiplicar.js**.

```
const mostrarTabla = () => {
    this.event.preventDefault();
    const numero = Number(document.getElementById('numero').value);
    if (numero >= 0 && numero <= 10) {

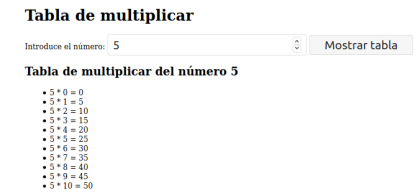
```

```
            let tabla = document.getElementById('tabla');
            let tablaMultiplicar = `<h2>Tabla de multiplicar del número ${numero}</h2>`;
            tablaMultiplicar += '<ul>';
            for (let i = 0; i <= 10; i++) {
                tablaMultiplicar += `<li>${numero} * ${i} = ${numero * i}</li>`;
            }
            tablaMultiplicar += '</ul>';
            tabla.innerHTML = tablaMultiplicar;
        } else {
            alert('El número introducido debe estar entre 0 y 10 (ambos inclusive');
            document.getElementById("numero").value = '';
        }
    }
```

Esta función realiza las siguientes acciones.

- Primero de todo previene que el evento se propague. Todo eso lo entenderás mejor más adelante, pero si no lo hiciese, al terminar limpia los campos y no nos deja ver el resultado (te animo a que comentes dicha línea -línea número 2- para que veas lo que sucede).
- Pasa el valor del campo de texto a número y luego comprueba mediante una sentencia condicional si está entre los valores aceptados o no.
  - Si lo está, rellena lo que se quiere mostrar en el bloque cuyo `id` es `tabla` mediante un bucle.
  - Si no lo está, alerta al usuario del error y elimina el contenido del campo de texto.

La visualización del mismo en el navegador es la que muestro a continuación, aunque te animo a que lo visualices tú en el tuyo.




Ministerio de Educación (CC BY-NC-SA)

# Autoevaluación

**El uso del bucle `for` en el ejemplo es el más adecuado. ¿Verdaero o falso?**

◯ Verdadero ◯ Falso

**Verdadero**

En este caso sabemos las iteraciones justas que debe dar el bucle, por que sí sería el más adecuado, aunque se podría haber llevado a cabo con cualquiera de los otros.

# 2.- Documentación, prueba y depuración de código JavaScript

## Caso práctico



Ministerio de Educación (Uso educativo nc)

**Ana**, una desarrolladora experimentada en **BK Programación**, aconseja a **Antonio** que continúe su formación enfocándose en las técnicas de documentación, prueba y depuración de código. Estas habilidades no solo le permitirán escribir código más limpio y eficiente, sino que también son fundamentales para la colaboración efectiva en equipo y el mantenimiento a largo plazo de las aplicaciones.

**Ana** advierte que, sin dominar estos conocimientos, **Antonio** podría enfrentar dificultades para integrarse en el desarrollo de proyectos colaborativos, ya que la capacidad de documentar y depurar código es crucial en cualquier entorno de desarrollo profesional.

En esta sección, exploraremos dos aspectos fundamentales del desarrollo de software en JavaScript: la documentación y la depuración de código. La documentación es esencial para crear un código claro y comprensible, facilitando la colaboración y el mantenimiento a largo plazo. Utilizaremos JSDoc, una herramienta que permite generar documentación a partir de comentarios en el código, similar a JavaDoc en Java. Por otro lado, la depuración es un proceso crucial para identificar y corregir errores en el código. Aprenderemos a utilizar el depurador del navegador para establecer puntos de parada, navegar por el código y observar el estado de las variables en tiempo real. Ambos temas son esenciales para mejorar la calidad y fiabilidad del código JavaScript.

# 2.1. Documentación JSDoc

JSDoc es una herramienta que permite generar documentación a partir de código JavaScript, similar a Java y JavaDoc. La documentación proporciona información sobre las funciones, variables y clases del código. Esta documentación se convierte en una guía de referencia para los desarrolladores que trabajan en el código base.

## Documentación del código JavaScript

Para documentar el código JavaScript es necesario preceder a la función o clase con el siguiente código:

```
/**
 *
 *
 */
```

En cada línea se añaden palabras reservadas para indicar los metadatos. Por ejemplo, si se quiere indicar el autor se utiliza **@author**.

```
/**
 * @author Óscar Gómez
*/
```

Vamos a ver algunos ejemplos simples de documentación y sus palabras reservadas:

### Variable

```
/**
 * El nombre de una persona.
 * @type {string}
 */
const name = "Oscar Gomez";
```

### Función

```
/**
 * Calcula la suma de dos números
 * @author Óscar Gómez
 * @param {number} num1 - El primer número.
 * @param {number} num2 - El segundo número.
 * @returns {number} La suma de los dos números.
```

```
 */
function suma(num1,num2){
    return num1+num2;
}
```

## Clase

```
/**
 * Clase que representa a un usuario.
 */
class User {
    /**
     * Crea un nuevo usuario.
     * @param {string} name - El nombre del usuario.
     * @param {number} age - La edad del usuario.
     */
    constructor(name, age) {
      /**
       * El nombre del usuario.
       * @type {string}
       */
      this.name = name;

      /**
       * La edad del usuario.
       * @type {number}
       */
      this.age = age;
    }

    /**
     * Obtiene una representación en cadena del usuario.
     * @returns {string} La representación en cadena del usuario.
     */
    getInfo() {
      return `${this.name} tiene ${this.age} años.`;
    }
}
```

## Debes conocer

Puedes ver una lista de todas las palabras reservadas para la generación de la documentación JsDoc en:

- JsDoc. Documentacion básica.
- Documentación de código con JSDoc (inglés).

# Generar la documentación

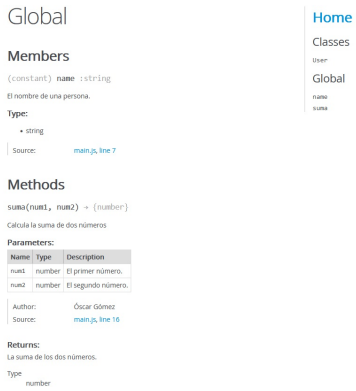
Para empezar a utilizar hay que instalar el paquete ejecutando el siguiente comando desde la terminal (con permisos de administrador):

```
npm install -g jsdoc
```

Ahora para generar la documentación de un archivo:

```
jsdoc .\js\main.js
```

La documentación se genera dentro de la carpeta **./out/** y para verla basta con abrir el archivo html de **main.js.html.**




Óscar Gómez. *JsDoc. Ejemplo de uso.* (Dominio público)

Por otro lado, para generar la documentación de todo el proyecto recursivamente en el directorio **./docs/** se debe ejecutar:

```
jsdoc . -r -d .\docs
```

## Autoevaluación

Para indicar el autor de un script se utiliza la etiqueta **@author**.

○ Verdadero ○ Falso

**Verdadero**

# 2.2. Prueba y depuración

Detectar errores en JavaScript a veces es complicado al ser un lenguaje interpretado. Una forma de estudiar por qué nuestro código no funciona es analizar el flujo de ejecución del código con el depurador. A continuación, un breve resumen de los principales conceptos del depurador:

- **Puntos de parada:** Los puntos de parada (o breakpoints) son marcadores que puedes establecer en tu código para detener la ejecución en un punto específico. Esto te permite inspeccionar el estado del programa en ese momento.
- **Panel de navegación:** El panel de navegación te permite moverte por el código mientras se está ejecutando. Puedes avanzar paso a paso (step over), entrar en funciones (step into) o salir de ellas (step out).
- **Estado de las variables:** Durante la depuración, puedes observar el valor de las variables en tiempo real. Esto es útil para entender cómo cambian a lo largo de la ejecución y detectar posibles errores.

## Código de ejemplo (Cálculo de la media de una lista de números)

Vamos a hacer un pequeño ejemplo utilizando el depurador del navegador Firefox. Para ello, partiremos del siguiente código que busca realiza la media de una lista de números mediante la suma de todos los números y, por último, dividiendo por el número de elementos en la lista.

```
1  //Definimos las notas
2  let notas=[5,7.5,4];
3
4  //Calculamos la media
5  let suma=0;
6  for(let n of notas){
7      suma*=n;
8  }
9
10 //Mostramos la media
11 console.log(`La media de [${notas}] es ${suma/notas.length});
```

Sin embargo, este código presenta un error que deberemos depurar. Este código actualmente devuelve como media 0, cuando debería ser 5.5:

```
La media de [5,7,4] es 0
```

## Pasos generales para depurar

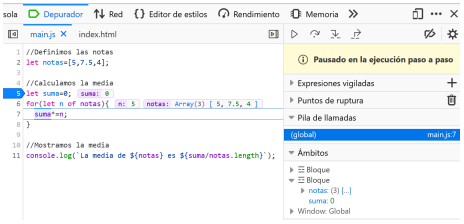
Para depurar este código, sigue estos pasos:

- **Abre el depurador.** Abre las herramientas del desarrollador (F12) y ve a la pestaña de depurador.
- **Establece un punto de parada:** Coloca un punto de parada en la línea donde comienza el bucle `for`. Esto te permitirá detener la ejecución antes de que se sumen las notas.




Óscar Gómez. *Depurador. Establecer puntos de ruptura.* (Dominio público)

- **Ejecuta el código:** Inicia la ejecución del script (por ejemplo, recargando la página) y observa cómo se detiene en el punto de parada. Usa las opciones de "step over" para avanzar línea por línea y verificar que la suma se está realizando correctamente.
- **Observa el estado de las variables:** Mientras avanzas, observa el valor de `suma` y `n` en cada iteración del bucle. Asegúrate de que están tomando los valores esperados.




Óscar Gómez. *Depurador Observar el estado de las variables* (Dominio público)

- **Detecta y corrige el error:** En este ejemplo, al entrar en el bucle podemos ver que el valor de la suma no se actualiza en el bucle. Esto nos permite deducir que el error está en dicha línea y en este caso se debe a que estamos utilizando el operador equivocado. ¡Estamos multiplicando en lugar de sumando!

El código arreglado sería:

```
1  //Definimos las notas
2  let notas=[5,7.5,4];
3
4  //Calculamos la media
5  let suma=0;
6  for(let n of notas){
7      suma+=n;
8  }
9
10 //Mostramos la media
11 console.log(`La media de ${notas} es ${suma/notas.length}`);
```

## Para saber más

También es posible realizar la automatización de pruebas con Mocha:

https://es.javascript.info/testing-mocha

# 3.- TypeScript

## Caso práctico



Ministerio de Educación (Uso educativo nc)

**Antonio** escucha a varios compañeros hablar sobre TypeScript y su creciente adopción en la empresa.

Escucha de refilón que permite introducir tipos a JavaScript y, con el apoyo de **Ana,** decide continuar su formación realizando varios proyectos de prueba en TypeScript.

TypeScript (TS) es un lenguaje de programación construido a un nivel superior de JavaScript (JS). Esto quiere decir que TypeScript dota al lenguaje de varias características adicionales (como la posibilidad de tipar variables) que hacen que podamos escribir código con menos errores, más sencillo, coherente y fácil de probar, en definitiva, más limpio y sólido.

TypeScript (Todos los derechos reservados)

En esta sección vamos a hacer una breve introducción a TypeScript para que conozcas sus características, diferencias con JavaScript y como hacer un hola mundo en TypeScript.

El código TypeScript es compilado a código JavaScript, que es el que finalmente es interpretado por el navegador.

## Para saber más

- Typescript lang.
- TypeScript para programadores JavaScript.
- Tipos en TypeScript.

# 3.1.- Conceptos básicos

TypeScript (TS) y JavaScript (JS) son lenguajes de programación ampliamente utilizados en el desarrollo web. Aunque TS se basa en JS, añade características adicionales que facilitan la escritura y mantenimiento del código, especialmente en proyectos grandes. A continuación, se presenta una comparativa simplificada de algunas características clave entre ambos lenguajes.

## Tipado

TypeScript es un lenguaje fuertemente tipado que permite crear tipos básicos, genéricos y enumerados. Los principales tipos son: **string, number, boolean y any** (equivalente a una variable normal no tipada de JavaScript). En este enlace puedes profundizar más sobre los tipos de JavaScript.

```typescript
// Ejemplo de tipado estático en TypeScript
let num: number = 10;
let str: string = "Hello";
```

## Clases

TypeScript simplifica y extiende las funcionalidades de trabajo con clases de JavaScript, con una sintaxis similar a Java o C#.

```typescript
// Ejemplo de clase en TypeScript
class Animal {
    private name: string;

    constructor(name: string) {
        this.name = name;
    }

    public move(distance: number): void {
        console.log(`${this.name} moved ${distance} meters.`);
    }
}

class Dog extends Animal {
    bark() {
        console.log("Woof! Woof!");
    }
}
```

# Interfaces

JavaScript no tiene soporte para interfaces. En TypeScript, las interfaces son esenciales para escenarios avanzados.

```
// Ejemplo de interfaz en TypeScript
interface Person {
    firstName: string;
    lastName: string;
    sayHi: () => string;
}

let customer: Person = {
    firstName: "Tom",
    lastName: "Hanks",
    sayHi: () => "Hi there"
};
console.log(customer.sayHi());
```

# Tuplas

JavaScript no soporta tuplas, parejas de datos, mientras que TypeScript sí lo hace.

```
// Ejemplo de tupla en TypeScript
let tuple: [number, string];
tuple = [10, "Hello"];
```

# Decoradores

JavaScript no tiene soporte para decoradores, metadatos, mientras que TypeScript sí.

```
@readonly
let name: string;
```

## Recomendación

Puedes hacer pruebas de TypeScript utilizando el compilador en línea.

# 3.2.- Puesta en marcha

Para instalar el compilador de TypeScript hay que ejecutar en la terminal (con permisos de administrador):

```
npm install -g typescript
```

Puedes comprobar que se ha instalado bien comprobando la versión que se ha instalado:

```
tsc --version
```

## Debes conocer

Si el comnado `tsc` no funciona, puede deberse a que no tenemos permisos de ejecución de scripts en PowerShell. Se soluciona modificandolo con:

```
Set-ExecutionPolicy undefined -Scope CurrentUser
```

# 3.3.- Hola mundo.ts

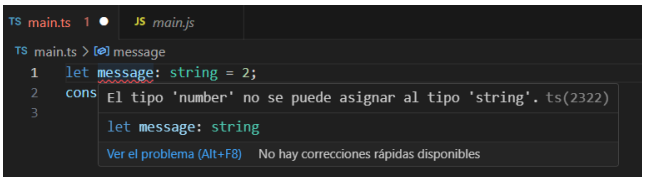
Los archivos TypeScript se nombran con la extensión **.ts**. Por ejemplo, **main.ts**.

```
let nombre:string = "Óscar Gómez";
console.log(`Hola ${nombre}`);
```

Ahora para compilarlo a código JavaScript basta con ejecutar:

```
tsc .\main.ts
```

Si hubiera algún error en el código TypeScript, no se podrá compilar el código y se mostrará la traza de los errores.




Óscar Gómez. *TypeScript. Ejemplo de error.* (Dominio público)

Ahora ya podemos ejecutar el código JavaScript, que se encuentra en **main.js** en el navegador o con node:

```
node.\main.js
//Muestra: Hola Óscar Gómez
```

## Para saber más

Puedes inicializar el proyecto con **tsc --init** y activar el autocompilado con **tsc -w**.

Además, en el archivo tsconfig, se indica donde se almacena el código JavaScript en la variable **outDir**.

## Autoevaluación

Es posible ejecutar código TypeScript directamente en el navegador web.

○ Verdadero ○ Falso

**Falso**

Falso. El código TypeScript tiene que ser compilado a código JavaScript para que pueda ser interpretado por el navegador web.

# 3.4.- Ejemplo proyecto TypeScript

En este apartado, se creará un pequeño proyecto utilizando TypeScript que consistirá en un formulario HTML que solicitará un nombre y, al hacer clic en un botón, mostrará un mensaje de saludo en pantalla.

## Estructura del proyecto

La estructura del proyecto se organizará de la siguiente manera:

```
/proyecto-typescript
│
├── /js
│   └── main.js
│
├── /ts
│   └── main.ts
│
├── index.html
└── tsconfig.json
```

## Código HTML

El archivo **index.html** contiene el esqueleto básico de la aplicación y enlaza el archivo JavaScript generado a partir de TypeScript:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>DWEC - Ejemplo proyecto TypeScript</title>
</head>
<body>
    <main>
        <div id="form">
            <input type="text" id="nombre" placeholder="Introduce tu nombre">
            <button id="saludarBtn">Saludar</button>
        </div>
    </main>
    <script src="./js/main.js" defer></script>
</body>
</html>
```

# Código TypeScript

El archivo **main.ts** dentro de la carpeta **ts** contiene la lógica de la aplicación. Se asegura de que, al cargar la página, se añada un evento al botón de saludo que recogerá el nombre introducido en el campo de texto y mostrará un mensaje con ese nombre.

```
//Inicialización
window.addEventListener("load",init);

function init(){
    document.querySelector("#saludarBtn")?.addEventListener("click",saludar);
}

function saludar(){
    let nombre: string = (<HTMLInputElement>document.querySelector("#nombre")).value ?? "Alicia"
    alert(`Hola ${nombre}`);
}
```

# Compilación del proyecto

Para compilar el código TypeScript a JavaScript, utiliza el siguiente comando en la terminal, situándote en la raíz del proyecto:

```
tsc ./ts/main.ts --outDir ./js
```

Este comando tomará el archivo **main.ts** de la carpeta **ts** y generará el archivo main.js dentro de la carpeta **js**. Una vez compilado, se podrá abrir el archivo **index.html** en un navegador y probar la funcionalidad.

Recuerda que, si hubiera algún error en el código TypeScript, no se podrá compilar y se mostrará un mensaje de error en la terminal.

## Para saber más

Para configurar la compilación automática de TypeScript cuando se realicen cambios en los archivos **.ts**, puedes usar el modo de "watch" que proporciona TypeScript. Esto permite que el compilador observe cambios en los archivos y vuelva a compilar automáticamente cuando detecte alguna modificación. Con esta configuración, el flujo de trabajo se vuelve más eficiente y asegura que siempre estés trabajando con la última versión compilada de tu código TypeScript.

## Configuración para Compilación Automática

Primero es necesario configurar el `tsconfig.json`: Si no lo has hecho ya, asegúrate de tener un archivo `tsconfig.json` en la raíz de tu proyecto. Este archivo se encarga de definir cómo se debe compilar tu proyecto TypeScript. El principal cambio que tendremos que realizar es modificar el valor de `outDir` para que apunte a la carpeta `./js`. Un ejemplo básico de `tsconfig.json` podría ser:

```
{
    "compilerOptions": {
        "target": "es6",                    // Version de ECMAScript para la compilación
        "outDir": "./js",                   // Directorio de salida para los archivos Jav
        "rootDir": "./ts",                  // Directorio raíz de los archivos TypeScript
        "strict": true                      // Activar todas las verificaciones estrictas
    },
    "include": [
        "./ts/**/*.ts"                      // Incluir todos los archivos TypeScript en e
    ]
}
```

Por último, hay que iniciar la compilación automática ejecutando el siguiente comando en la terminal:

```
tsc --watch
```

Este comando observará los archivos `.ts` en el directorio especificado y los volverá a compilar cada vez que detecte un cambio. Los archivos JavaScript resultantes se guardarán automáticamente en la carpeta de salida especificada (`./js` en este caso).

# 3.5.- Otros usos de Javascript

JavaScript no está limitado al uso de aplicaciones web. Se puede utilizar para realizar aplicaciones en el lado del servidor o incluso android.

- Node. El JavaScript en el lado del Servidor.
  - Tutorial de Node. Node Begginer.
- React Native. Permite desarrollar aplicaciones móviles utilizando JavaScript y React, ofreciendo rendimiento nativo en iOS y Android.
  - React Native

También es importante conocer herramientas que permiten el desarrollo de aplicaciones híbridas, diseñadas para web y móviles, tales como Flutter o Ionic.